Towards Robust Detection of Open Source Software Supply Chain Poisoning Attacks in Industry Environments

Xinyi Zheng*†‡
xinyizheng@hust.edu.cn
Huazhong University of Science and Technology
Wuhan, China

Chen Wei*
juyi.wc@mybank.cn
MYbank, Ant Group
Hangzhou, China

Shenao Wang†‡
shenaowang@hust.edu.cn
Huazhong University of Science and Technology
Wuhan, China

Yanjie Zhao†‡
yanjie_zhao@hust.edu.cn
Huazhong University of Science and Technology
Wuhan, China

Peiming Gao§
peiming.gpm@mybank.cn
MYbank, Ant Group
Hangzhou, China

Yuanchao Zhang
yuanchao.zhang@mybank.cn
MYbank, Ant Group
Hangzhou, China

Kailong Wang†§
wangkl@hust.edu.cn
Huazhong University of Science and Technology
Wuhan, China

Haoyu Wang†‡
haoyuwang@hust.edu.cn
Huazhong University of Science and Technology
Wuhan, China

arXiv:2409.09356v1 [cs.CR] 14 Sep 2024

ABSTRACT

The exponential growth of open-source package ecosystems, particularly NPM and PyPI, has led to an alarming increase in software supply chain poisoning attacks. Existing static analysis methods struggle with high false positive rates and are easily thwarted by obfuscation and dynamic code execution techniques. While dynamic analysis approaches offer improvements, they often suffer from capturing non-package behaviors and employing simplistic testing strategies that fail to trigger sophisticated malicious behaviors. To address these challenges, we present OSCAR, a robust dynamic code poisoning detection pipeline for NPM and PyPI ecosystems. OSCAR fully executes packages in a sandbox environment, employs fuzz testing on exported functions and classes, and implements aspect-based behavior monitoring with tailored API hook points. We evaluate OSCAR against six existing tools using a comprehensive benchmark dataset of real-world malicious and benign packages. OSCAR achieves an F1 score of 0.95 in NPM and 0.91 in PyPI, confirming that OSCAR is as effective as the current state-of-the-art technologies. Furthermore, for benign packages exhibiting characteristics typical of malicious packages, OSCAR reduces the false positive rate by an average of 32.06% in NPM (from 34.63% to 2.57%) and 39.87% in PyPI (from 41.10% to 1.23%), compared to other tools, significantly reducing the workload of manual reviews in real-world deployments. In cooperation with Ant Group, a leading financial technology company, we have deployed OSCAR on its NPM and PyPI mirrors since January 2023, identifying 10,404 malicious NPM packages and 1,235 malicious PyPI packages over 18 months. This work not only bridges the gap between academic research and industrial application in code poisoning detection but also provides a robust and practical solution that has been thoroughly tested in a real-world industrial setting.

CCS CONCEPTS

• **Security and privacy** → **Malware and its mitigation**; • **Software and its engineering** → **Software libraries and repositories**; **Open source model**.

KEYWORDS

OSS Supply Chain, Malicious Code Poisoning, PyPI, NPM

ACM Reference Format:
Xinyi Zheng, Chen Wei, Shenao Wang, Yanjie Zhao, Peiming Gao, Yuanchao Zhang, Kailong Wang, and Haoyu Wang. 2024. Towards Robust Detection of Open Source Software Supply Chain Poisoning Attacks in Industry Environments. In *39th IEEE/ACM International Conference on Automated Software Engineering (ASE '24), October 27-November 1, 2024, Sacramento, CA, USA.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3691620.3695262

1 INTRODUCTION

With the rapid development and widespread adoption of open-source software, package managers like the Node.js Package Manager (NPM) [28] and the Python Package Index (PyPI) [6] have become integral to modern software development workflows. These package managers enable developers to easily share, reuse, and build upon existing code, greatly accelerating development cycles and fostering collaboration. However, as the open-source ecosystem expands, the number of packages needing management is growing exponentially, with NPM and PyPI registries being updated at an unprecedented frequency [42]. This explosive growth has also given rise to an alarming trend of open-source software supply chain poisoning attacks [3, 17, 30]. Malicious actors are increasingly targeting these package managers to distribute compromised packages, which can introduce backdoors [30] or malware [17] into

*Both authors contributed equally to this research.
†Hubei Key Laboratory of Distributed System Security, Hubei Engineering Research Center on Big Data Security, School of Cyber Science and Engineering, Huazhong University of Science and Technology.
‡These authors are also affiliated with Wuhan Jinyinhu Laboratory.
§Kailong Wang (wangkl@hust.edu.cn) and Peiming Gao (peiming.gpm@mybank.cn) are the corresponding authors.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ASE '24, October 27-November 1, 2024, Sacramento, CA, USA

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1248-7/24/10
https://doi.org/10.1145/3691620.3695262

downstream applications that depend on them [19, 51]. The consequences of such attacks can be severe, ranging from data breaches and intellectual property theft to widespread system compromises and reputational damage.

Research Gaps. Traditional vetting and security measures applied to proprietary software (such as signature-based VirusTotal) are often less stringent [8, 17]. Effectively detecting malicious packages in real repositories to ensure the safety of the software supply chain remains a significant challenge [31, 45]. One significant limitation of static analysis methods is their high false positive rates when applied to real-world package repositories like NPM and PyPI [45]. These repositories are vast and continually expanding, with diverse coding styles, frameworks, and development practices. Static analysis tools, relying on predefined rules and patterns [2, 17, 23], struggle to adapt to this heterogeneity, leading to numerous false alarms. Consequently, security teams face overwhelming manual review and validation workloads [8, 45], hindering their ability to efficiently identify and respond to genuine threats. Another limitation in detecting malicious packages stems from the widespread use of code obfuscation [9, 26, 38] and dynamic code execution [8, 51] techniques in NPM and PyPI. These techniques, while sometimes used legitimately, create significant barriers for static analysis methods [3, 8]. Obfuscated code obscures the true intent and functionality of the software, making it challenging to discern benign from malicious behavior through static inspection alone. Dynamic execution techniques, such as runtime code generation or remote code loading, further complicate analysis by deferring the actual code execution until runtime. Existing dynamic detection methods, such as MalOSS [3], also face significant limitations despite their improvements over static approaches. System-level monitoring often captures non-package behaviors [9], introducing noise and potential false positives. Moreover, their package testing strategies are often simplistic, relying on basic function invocations and class initializations with null arguments. This approach may fail to trigger sophisticated malicious behaviors that depend on specific inputs or environmental conditions.

Our Work. To address these challenges, we propose OSCAR (Open-source Supply Chain Attack Reconnaissance), a robust dynamic code poisoning detection pipeline for NPM and PyPI ecosystems. Our key insight is to disregard the various sophisticated techniques used by malicious packages and focus on their specific behaviors. Attackers typically design their malicious actions to be easily executable, as overly complex or deeply nested activation conditions inherently reduce the potential for successful exploitation. This aspect of attack design makes dynamic detection particularly effective for analyzing packages. Our method fully executes malicious packages in a sandbox environment, including package installation and import. We also employ fuzz testing techniques [18, 27] on the exported functions/classes within the packages. Additionally, we implement aspect-based behavior monitoring methods for Node.js and Python processes, setting the most suitable API hook points based on long-term detection experience. Finally, we design a set of heuristic matching rules for behavior logs, tailored to identify different types of malicious activities. This customization significantly enhances our ability to accurately pinpoint and flag malicious packages.

To evaluate the effectiveness of OSCAR, we have constructed a benchmark dataset that includes real-world samples of malicious NPM and PyPI packages from industry activities and a dataset to detect risky benign packages with confusing characteristics. Using these datasets, we have compared OSCAR with six existing tools. Since its deployment in January 2023, spanning over a year and a half of comprehensive monitoring on Ant Group's NPM and PyPI mirror repositories, OSCAR has successfully identified 10,404 malicious NPM packages and 1,235 malicious PyPI packages. These findings have prompted Ant Group to remove the harmful packages from the mirror sources, enhancing the security of these package management systems. Before removal, we have collected and classified the malicious packages, which have been made publicly available at https://github.com/security-pride/OSCAR.

We summarize the main contributions of this paper as follows:

- **Robust Pipeline.** We introduce OSCAR, a novel dynamic analysis pipeline for detecting malicious packages in NPM and PyPI ecosystems. OSCAR overcomes the limitations of static analysis methods by fully executing packages in a sandboxed environment, employing fuzz testing on exported functions and classes, and implementing aspect-based behavior monitoring for Node.js and Python processes.
- **Outstanding Performance.** We present a comprehensive evaluation of OSCAR against six existing tools using a benchmark dataset of real-world malicious packages and benign packages. OSCAR achieves an F1 score of 0.95 in NPM and 0.91 in PyPI, indicating that OSCAR is as effective as SOTA techniques. Furthermore, for benign packages exhibiting characteristics typical of malicious packages, OSCAR reduces the false positive rate by an average of 32.06% in NPM (from 34.63% to 2.57%) and 39.87% in PyPI (from 41.10% to 1.23%). Our results demonstrate OSCAR's superior effectiveness in detecting sophisticated malicious behaviors while significantly reducing false positives compared to current SOTA techniques.
- **Industrial Deployments.** We demonstrate the practical effectiveness of OSCAR through a large-scale, long-term deployment on Ant Group's NPM and PyPI mirrors. Over a period of more than 18 months, OSCAR successfully identified and intercepted 10,404 malicious NPM packages and 1,235 malicious PyPI packages in real-world industrial settings. We offer a detailed classification and analysis of these malicious packages, contributing valuable data to the research community for future research.

2 BACKGROUND

2.1 Open-Source Software Supply Chain Attacks

The primary attack vectors in contemporary open-source software supply chains include *Typosquatting* [36, 43], *Dependency Confusion* [15, 22], and *Account Compromise* [13, 15, 36, 48]. Attackers exploit these attack vectors to publish malicious packages on platforms, utilizing unique activation conditions within the supply chain to initiate malicious activities. Attacks involving malicious packages can be categorized based on when the attack logic is executed. Typically, the malicious code is activated during one of three

stages: install-time (when the package is being installed), import-time (when the package is being imported into another program), or run-time (when the package's code is being executed) [3, 8].

Table 1: Three types of execution stages and activation conditions of malicious code poisoning attacks.

Execution Stage	Entry Point	Activation Condition
Install-Time	setup.py	pip install pkg
	"preinstall" or "postinstall" in package.json	npm install pkg
Import-Time	__init__.py	import pkg
	"main" in package.json	require(pkg)
Run-Time	in any .py file	traversal trigger
	in any .js file	traversal trigger

As shown in Table 1, both Install-Time Attacks and Import-Time Attacks rely on the installation and dependency resolution mechanisms of package managers. These attacks inject malicious logic into specific scripts, which are automatically triggered when the corresponding package is installed or imported. In the case of PyPI packages, the `setup.py` file is executed during installation, while the `__init__.py` file is executed when the package is imported. In NPM packages, the `package.json` file defines the execution of code at various stages. Commands specified in the `preinstall` field are executed before package installation, while those in the `postinstall` field run after installation is complete. When the package is imported into another program, the file specified in the `main` field is executed. Attackers frequently utilize these easily triggered methods to ensure the success of their malicious activities. However, some targeted attacks prioritize stealth by embedding malicious code within specific functions or methods in secondary files, rather than in the main file. These attacks, known as run-time attacks, only activate when the compromised function or method is called during the application's execution. By hiding the malicious code in less prominent locations and triggering it only under specific circumstances, these attacks can be particularly difficult to detect through preemptive measures. The above-mentioned execution stages and activation conditions collectively depict the intricate security threats present in the open-source software supply chain.

2.2 Code Poisoning Detection

Currently, the detection of malicious code in the open-source software supply chain primarily depends on static analysis techniques. These techniques include both rule-based and machine learning-based tools [8, 31, 45]. **Rule-based tools** employ a set of predefined heuristic rules or patterns to characterize suspicious behaviors or code structures, thereby facilitating the identification of malicious code, such as Guarddog [2]. While this method proves highly effective in specific contexts, its success critically hinges on the quality and completeness of the rule set, necessitating expert knowledge for its development and ongoing maintenance. Moreover, this approach is susceptible to a high incidence of false positives. Conversely, **machine learning-based tools** analyze software packages using algorithms to extract pivotal features, which are then utilized to train models capable of differentiating between benign and malicious activities. Amalfi [37] and SAP [16] are representative tools. The efficacy of these tools is contingent upon the quality and variety of the training data. Additionally, they are prone to manipulation by techniques designed to circumvent the underlying learning models.

There are limited **dynamic detection tools** available, with MalOSS [3] serving as a representative tool. Tools based on dynamic detection execute software packages in a sandbox to monitor behavior and analyze data to detect malicious activities. Existing dynamic detection tools frequently lack comprehensive coverage of execution paths, increasing the risk of overlooking highly covert malicious packages.

3 MOTIVATION EXAMPLE

Obfuscation is a common technique used in malicious packages found in package managers like NPM and PyPI. As shown in Figure 1, the index.js file of the NPM package discord.js-selbotderank-1.0.0 is heavily obfuscated. After de-obfuscation, it reveals code that sends Discord user authorization tokens, IDs (line 16), and usernames (line 13) to an external server (line 20). Similarly, using network communication services to exfiltrate information or download executable malicious code is another common trait of malicious packages.However, these techniques are also frequently found in benign packages. According to statistics from Socket [40], many benign packages exhibit features such as obfuscated code, network access, and shell access. Therefore, relying solely on these features to identify malicious packages is not reasonable.

Extracted from discord.js-selbotderank-1.0.0/index.js

```
1  ...const axios=require(_0x27316a('0x462',0x464,'D0sx',0x3b7,'0
x564'));exports[_0x10b8f9('0x239','0x15e',0x1d,'bq7w','0x133')]=f
unction(_0x44568e){const _0x5f4605={_0x33d8b9:0x152,...
```

Deobfuscation

```
1  const axios = require("axios");
2  exports.Start = function(authToken) {
3   return new Promise(async (resolve, reject) => {
4    try {
5     const userResponse = await axios.get(
6      "https://discord.com/api/v9/users/@me",
7      { headers: { "Content-Type": "application/json",
           ↪Authorization: authToken } }
8     );
9     const userData = userResponse.data;
10    if (userData.message !== "401: Unauthorized") {
11     const webhookUrl = "http://194.87.xxx.xxx:3005/noleak";
12     const embed = { color: 3092790,
13      title: userData.username + "#" + userData.discriminator,
14      url: "https://discord.id/?prefill=" + userData.id,
15      author: {name: "dmall logs", icon_url: "...",
           ↪url: "https://discord.id/?prefill=" + userData.id },
16      description: authToken + "\n\n<@" + userData.id + ">",
17      thumbnail: { url: "..." }
18     };
19     const payload = { embeds: [embed] };
20     await axios.post(webhookUrl, payload);
21    ...
```

Figure 1: Obfuscated JavaScript malicious package example.

Current static analysis methods, including both machine learning-based and rule-based approaches, mainly rely on analyzing metadata and source code features to detect malicious packages. However, this approach often leads to high false positive rates, as it struggles to differentiate between benign and malicious packages that share similar features.

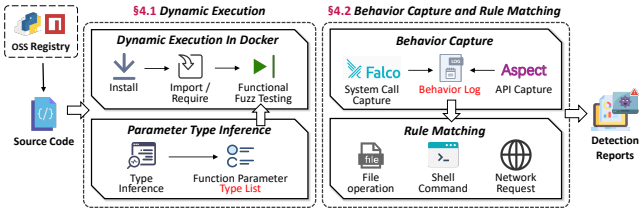


Figure 2: The workflow of OSCAR, which executes packages in docker (§ 4.1), encompassing package installation, import, and function/class invocation, followed by suspicious behavior capture using AOP and Falco (§ 4.2).

4 OSCAR WORKFLOW

The overview of OSCAR is shown in Figure 2. We will introduce these modules in detail in the following sections.

4.1 Dynamic Execution

As discussed in § 2.1, the majority of package attack activations occur during the installation, import, and execution stages. To identify potential security issues, the dynamic execution module focuses on these stages. In particular, we sequentially execute each of the three stages within a controlled environment, allowing us to monitor for any suspicious or malicious activity that may indicate a package attack. To this end, we design two Docker images based on the official Node.js image [10] and Python image [11], deploying dynamic execution configurations and monitoring tools within them. Each package is executed within an independent Docker container, following the steps below to ensure comprehensive coverage of the activation stages.

- **Package Installation.** To minimize external traffic communication during package execution, we upload the package source code into the Docker container for local installation.
- **Package Import.** We traverse all modules within the package installation path and import them.
- **Function Invocation / Class Initialization.** To detect highly concealed attacks as comprehensively as possible, we implement a function-level fuzz testing method by recursively traversing the declared objects within the imported modules and executing all functions and methods within classes.

The inspiration for OSCAR comes from analyzing the behavior of malicious packages. Attackers often design their malicious code to activate during the installation and import stages. This ensures that the attack is successfully triggered when the package is deployed. However, some more sophisticated attacks prioritize stealth and concealment. In these cases, the malicious logic is buried deep within a function defined inside the package, making it harder to detect. By systematically testing every function and method within the imported modules, our function-level fuzz testing method can uncover even the most well-hidden malicious code.

The implementation of our function-level fuzz testing is described by Algorithm 1. Initially, we organize all imported module objects into the dictionary `packageExportMap` with key-value pairs of `module_name` and `require(module_name)`, which we then pass to the `fuzz` module for processing (line 22). The `fuzz` module (line

Algorithm 1: Function-level Fuzz Testing

```
   Input: packageExportMap
   Output: No explicit return value
1  Function fuzz(object, depth):
2      type ← getType(object);
3      if type is Function or type is Class then
4          handleClass(object.constructor, object);
5      else if type is Object and depth < 2 then
6          handleObject(object, depth);
7  Function handleClass(cls, object):
8      parser ← classParser(cls, currentPath);
9      if parser.isClass() then
10         invokeStaticMethods(parser.staticMethods);
11         invokeInstanceMethods(cls, parser.methods,
               object);
12     else
13         invokeFunction(cls);
14     end
15 Function handleObject(object, depth):
16     rawPath ← currentPath;
17     foreach key ∈ object do
18         currentPath ← currentPath + [key];
19         fuzz(object[key], depth + 1);
20         currentPath ← rawPath;
21     end
22 fuzz(packageExportMap, 0);
```

1) processes inputs based on their type. For functions, we directly invoke them using constructed parameters (line 13). For classes, static methods are called directly (line 10), whereas instance methods necessitate the instantiation of an object prior to invocation (line 11). Additionally, if the object is of type `Object`, each constituent element is subject to recursive processing facilitated by the `fuzz` module (line 19). It is important to note that the global variable `currentPath` (line 16), mentioned in the algorithm, always aligns with the path of the module currently under analysis.

Dynamically triggering each function or method presents a significant challenge: constructing the appropriate input parameters. The success of a function call heavily relies on passing the right parameters in the correct format. To overcome this obstacle, we employ static type inference tools [12, 25] to analyze the package's source code before conducting fuzz testing. During this analysis phase, we extract and save crucial information about all functions and class methods, including parameter names, default values, and parameter types. By gathering this data and utilizing it alongside randomly generated initialization seeds, we can construct more effective initial parameters for function or method calls. This approach significantly boosts the likelihood of successfully executing and covering the targeted functions or methods during our fuzz testing process, thoroughly exploring the package's behavior and uncover potential vulnerabilities or hidden malicious logic.

4.2 Behavior Capture and Rule Matching

When executing packages in the sandbox, based on insights from previous studies[3, 33], we monitor three main aspects: network behavior, file behavior, and process behavior. Network behavior

Table 2: Pointcuts in JavaScript for malicious code detection.

Category	Lib	API
Network	net.Socket.prototype	connect
	dgram.Socket.prototype	connect, send
	dns, dns.promises	lookup, lookupService
	_http_outgoing.OutgoingMessage.prototype	_flushOutput, _writeRaw
	dns, dns.promises, dns.Resolver .prototype, dns.promises .Resolver.prototype	resolve, resolve4, resolve6, resolveAny, resolveCaa, resolveCname, resolveMx, resolveNaptr, resolveNs, resolvePtr, resolveSoa, resolveSrv, resolveTxt, reverse
	_http_client.ClientRequest.prototype	onSocket
File	fs	readFile, readFileSync, rmdir, rmdirSync, unlink, unlinkSync, writeFile, writeFileSync, rename, renameSync
Process	child_process.ChildProcess.prototype	spawn
	child_process	execSync, execFileSync, spawnSync

Table 3: Pointcuts in Python for malicious code detection.

Category	Lib	API
Network	socket	create_connection, getaddrinfo, gethostbyname, gethostbyname_ex
	socket.socket	connect_ex, sendto, send, sendmsg, sendall, connect
	pycares.Channel	getaddrinfo, query, search
	aiohttp.client_reqrep .ClientRequest	write_bytes
	http.client .HTTPConnection	_send_request, putrequest, send
	urllib3.connection .HTTPConnection	request, request_chunked
	httpcore._backends .sync.syncStream	write
	httpcore._sync .connection .HTTPConnection	handle_request
File	os	rmdir, remove, unlink, read, readv, write, writev, open, rename, replace
	builtins	open
	shutil	rmtree
	io.BufferedReader, io.BufferedRandom	readinto, readinto1, read, readlines, read1, readline
	io.BufferedWriter, io.BufferedRandom	write
Process	os	system, posix_spawn, posix_spawnp, _execvpe, execv, execve
	subprocess.Popen	__init__

monitoring focuses on external TCP, UDP communications, and DNS requests. File behavior monitoring targets file read/write operations, deletions, links, and renames. Process behavior monitoring focuses on process creation events. To achieve comprehensive monitoring, we implement strategies at both the API and system call levels. In collaboration with security experts from Ant Group and through the comprehensive analysis of existing security reports on software supply chain poisoning attacks, we have compiled a list of critical APIs in Python and JavaScript that are highly effective for detecting malicious packages. We further categorize these APIs according to the three types of behavior mentioned above, as shown in Table 2 and Table 3.

In our implementation, we employ an aspect-based method to monitor API behavior. We define pointcuts, which are specific points in the program, such as method calls or field accesses. These pointcuts are associated with the APIs we intend to monitor, triggering when these APIs are invoked. The advice, which consists of actions executed at these pointcuts, captures and logs the method name and parameter information. This approach enables efficient monitoring and logging of API behavior. This method is implemented based on the AOP (Aspect-Oriented Programming) [14] technology. AOP supports weaving aspects into the target code at runtime. Through runtime weaving, we can dynamically add or modify pointcuts and advice during program execution without stopping or recompiling the program. This capability allows the system to flexibly adjust monitoring logic according to changing requirements. Compared to the API Hook method that requires static binding, this flexibility is crucial for long-term, large-scale real-time monitoring. Additionally, attackers sometimes use custom functions to replace common library APIs to evade detection. To counter this technique, we use Falco [5] to monitor network, file, and process behaviors at the system level. This multi-layered monitoring strategy ensures comprehensive detection of all potential malicious activities.

Both the aspect-oriented method and Falco generate detailed logs, recording all captured API calls and system behaviors. We determine whether a package is malicious based on these logs and predefined black and white list rules. The white list is used to exclude interference from local services, such as requests to local servers, modifications to temporary directories, and npm process launches. The black list focuses on network, file, and command execution behaviors, screening unknown IPs, malicious domains, sensitive information (e.g., passwords), sensitive files (e.g., `/etc/passwd`, `bashrc`), and the execution of sensitive processes (e.g., `nc`, `chmod`). Except for network activities that transmit personal or sensitive

information to unknown domains, which require manual review, packages that match black list rules are generally deemed malicious.

5 EVALUATION

Our evaluation targets the following research questions:

- **RQ1 (Performance).** How does OSCAR perform in terms of code poisoning detection on the benchmark dataset?
- **RQ2 (Improvements).** What advancements does OSCAR provide compared to other state-of-the-art techniques?
- **RQ3 (Industrial Deployment).** Can OSCAR effectively detect malicious packages in large-scale real-world data?

5.1 Datasets

Benchmark Dataset. We construct a benchmark dataset containing both malicious and benign packages. The NPM malicious packages originate from those published on Snyk [39], OSV [35], and Socket Security [41] between January 2024 and June 2024. The PyPI malicious packages are randomly selected from the pypi_malregistry [20] dataset, where most of the malicious packages are uploaded recently. The benign packages are randomly chosen from popular packages. In both NPM and PyPI, the numbers of malicious and benign packages are set to 500 and 1500, respectively, ensuring a consistent 1:3 ratio. We evaluate the selected artifacts and tools on this benchmark dataset to assess their ability to correctly identify malicious packages (RQ1).

Risk-Characteristic Benign Dataset. Additionally, to demonstrate the improvements of OSCAR over existing static detection tools, we collect a set of benign packages with risk characteristics but without actual malicious behavior from Socket [40]. These packages, downloaded from official repositories, are specifically chosen to challenge and compare the accuracy of OSCAR against current static analysis methods. This process result in two datasets of benign packages with risk characteristics. The first dataset contains obfuscated features (OBF), where each package includes at least one `.py` or `.js` file with obfuscated content. This dataset comprises 181 PyPI packages and 594 NPM packages. The second dataset contains features similar to remote download and execution behavior (RDE), where each package includes at least one `.py` or `.js` file with network download requests and process execution behavior. This dataset comprises 223 PyPI packages and 515 NPM packages. By analyzing these packages, we aim to highlight OSCAR's superior ability to differentiate between truly malicious behavior and benign code that merely exhibits risk-like characteristics, thus reducing false positives and improving overall detection efficacy (RQ2).

5.2 Baseline Selection

To ensure a comprehensive analysis, we systematically review the literature and associated studies from the past three years, focusing on tools designed to detect malicious code in NPM and PyPI packages. For our benchmark tests, we adopt the following selection criteria for these tools:

1) Open Source: An in-depth understanding of the detection techniques employed by the tools is essential, particularly in terms of which types of malicious packages are prioritized or neglected during the scanning process. Consequently, we restrict our selection to tools that are fully open-source or have accessible source code.

2) Detection Granularity: Certain tools restrict their detection capabilities to merely scanning metadata or the architecture of package managers when identifying third-party malicious packages. However, our study exclusively considers tools that perform analyses on the source code of the packages.

3) Fully Automated: Our research prioritizes tools that operate in a fully automated manner, necessitating no further customization or training.

Table 4: The selected representative artifacts capable of detecting malicious code poisoning in CI/CD pipelines.

Baseline	Language	Technique used
SAP [16]	Python, JavaScript	Static (ML)
Amalfi [37]	JavaScript	Static (ML)
Bandit4Mal [21]	Python	Static (Rule)
OSSGadget [24]	Python, JavaScript	Static (Rule)
AppInspector [23]	Python, JavaScript	Static (Rule)
Guarddog [2]	Python, JavaScript	Static (Rule)

Following these principles, we shortlist six representative baseline artifacts capable of detecting malicious code poisoning in CI/CD pipelines: SAP [16], Amalfi [37], Bandit4Mal [21], OSSGadget [24], AppInspector[1] [23], and Guarddog [2]. These tools are all static analysis tools. SAP and Amalfi are machine learning-based, while the other four tools are rule-based. Among them, Bandit4Mal only supports Python, and Amalfi only supports JavaScript. The other tools support both JavaScript and Python.

5.3 Performance (RQ1)

We evaluate OSCAR and six other baseline artifacts on the dataset containing both malicious and benign packages. We use true positives (TP, the number of packages correctly classified as malicious), false positives (FP, the number of packages incorrectly classified as malicious), and false negatives (FN, the number of packages incorrectly classified as benign) to assess performance. Additionally, to measure overall effectiveness, we use precision, recall, and F1 score as evaluation metrics.

Our experiment focuses on determining if a package is malicious, not on detailed analysis results. Here is how we classify packages as malicious for each tool. For SAP and Amalfi, results directly indicate the package's nature. For Guarddog and OSCAR, any matching report deems a package malicious. For Bandit4Mal, OSSGadget, and AppInspector, each rule is accompanied by a Severity and Confidence score. We classify a package as malicious if both the severity and confidence are high for Bandit4Mal and AppInspector. For OSSGadget, a package is malicious if both the severity and confidence are above medium[2]. Additionally, during the detection process, all tools utilize the default configurations for their respective rules and feature sets.

Performance Comparison. The detection results are shown in Table 5. Overall, OSCAR performs excellently on both datasets, particularly in terms of precision and F1 scores. On the PyPI dataset,

[1]We use AppInspector to represent ApplicationInspector for short.
[2]These thresholds were determined through comprehensive analysis to optimize detection accuracy while minimizing false positives across benchmark datasets.

Table 5: Evaluation results on the benchmark dataset.

Artifact	TP	FP	FN	Pre.	Rec.	F1
SAP	431	163	69	0.73	0.86	0.79
Bandit4Mal	109	208	391	0.34	0.22	0.27
OSSGadget	119	98	381	0.55	0.24	0.33
AppInspector	92	664	408	0.12	0.18	0.15
Guarddog	**472**	61	**28**	0.89	**0.94**	**0.91**
OSCAR	423	**4**	77	**0.99**	0.85	0.91

(a) PyPI

Artifact	TP	FP	FN	Pre.	Rec.	F1
SAP	418	45	82	0.90	0.84	0.87
Amalfi	428	17	72	0.96	0.86	0.91
OSSGadget	262	147	238	0.64	0.52	0.58
AppInspector	237	425	263	0.36	0.47	0.41
Guarddog	338	28	162	0.92	0.68	0.78
OSCAR	**459**	**3**	**41**	**0.99**	**0.92**	**0.95**

(b) NPM

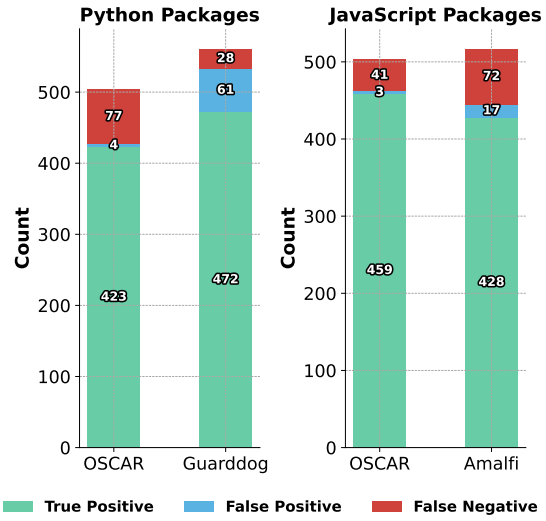


Figure 3: Compare results on the benchmark dataset.

OSCAR achieves the highest precision at 0.99, compared to Guarddog's 0.89. Although Guarddog has a higher recall rate of 0.94 compared to OSCAR's 0.85, OSCAR's F1 score is 0.91, matching Guarddog. On the NPM dataset, OSCAR achieves the highest precision at 0.99, surpassing Amalfi's 0.96. OSCAR also achieves the highest recall rate of 0.92, compared to Amalfi's 0.86, resulting in an F1 score of 0.95, higher than Amalfi's 0.91. These results highlight OSCAR's outstanding ability to detect sophisticated malicious behaviors while maintaining high precision and F1 scores, effectively reducing false positives and manual review workload. In terms of time efficiency, OSCAR achieves an average testing time of 165 seconds for pypi packages and 128 seconds for npm packages in a single-threaded environment. In practical applications, we use multiple devices and processes for concurrent monitoring, ensuring that the testing time remains within an acceptable range. Thus, OSCAR's performance is sufficient to meet the detection requirements.

Figure 3 provides a more intuitive display of the detection results, showing specific TP, FP, and FN data. In PyPI, compared to Guarddog, OSCAR has more false negatives but fewer false positives. Analyzing Guarddog's detection reports, we find that over 70% of its true positives come from its cmd-overwrite rule matching rewrites of the install method in the `setup.py` file, without correctly identifying other malicious behaviors. In NPM, compared to Amalfi, OSCAR has fewer false positives and false negatives.

We analyze the false negatives of OSCAR and summarize the following reasons. First, OSCAR's sandbox environment is Linux-based, so it cannot detect attacks targeting Windows or other systems. Second, some malicious packages use anti-sandbox techniques against dynamic detection, which OSCAR currently cannot cover. Additionally, for some malicious logic hidden inside functions with complex parameters, we cannot construct suitable initial parameters to successfully execute the functions and trigger the malicious logic. Furthermore, some PyPI malicious packages are from earlier sources, and version mismatches may lead to installation failures, resulting in false negatives.

Root Cause Analysis of Detection Performance. Additionally, combining the detection results and the characteristics of each tool, we draw several conclusions. Guarddog, OssGadget, and AppInspector directly match rules against source code, with Guarddog also considering metadata. Bandit4Mal generates an Abstract Syntax Tree (AST) from the code and matches APIs on the AST. However, results show that detection capability ultimately depends on the rules defined. Additional semantic analysis does not significantly enhance detection capability. Comprehensive rules increase false positive rates, while limited rules increase false negative rates. SAP and Amalfi extract features from source code files, including not only code content but also word counts and entropy values. Machine learning methods show better results as they consider multiple factors rather than relying on single rules. However, the drawback is that overly comprehensive features can lead to false negatives for large malicious packages and false positives for small benign packages, as most existing malicious packages are small. OSCAR identifies malicious packages through runtime behavior monitoring and behavior content rule matching. Due to long-term dynamic monitoring, OSCAR's behavior content matching rules are well-developed, resulting in superior performance compared to other tools. However, the dynamic nature of OSCAR leads to a high number of false positives, which requires improvement.

ANSWER to RQ1

OSCAR matches or exceeds the performance of SOTA tools in both NPM and PyPI, demonstrating particularly high accuracy in detection.

5.4 Improvement (RQ2)

We evaluate OSCAR and six other baseline artifacts on the risk-characteristic benign dataset. The evaluation metric is the false

positive rate (FPR), which indicates the proportion of benign packages incorrectly marked as malicious by each tool. The detection results are shown in Table 6, where OBF represents the dataset with obfuscation features and RDE represents the dataset with remote download and execution-like features.

Table 6: FPR results on the risk-characteristic benign dataset.

Artifact	FPR (PyPI)		FPR (NPM)	
	OBF	RDE	OBF	RDE
SAP [16]	36.46%	21.08%	12.12%	5.05%
Amalfi [37]	-	-	6.23%	17.09%
Bandit4Mal [21]	34.25%	58.30%	-	-
OSSGadget [24]	34.81%	22.42%	42.76%	45.83%
AppInspector [23]	82.87%	83.86%	67.00%	84.47%
Guarddog [2]	12.71%	24.22%	34.85%	28.93%
OSCAR	**1.10%**	**1.35%**	**3.20%**	**1.94%**

Based on the experimental results, OSCAR demonstrate a significant reduction in false positive rates (FPR) when detecting NPM and PyPI packages compared to other tools. In NPM, the average FPR of other tools in the OBF and RDE scenarios are 32.99% and 36.27%, respectively. In comparison, OSCAR's FPRs in these scenarios are 3.20% and 1.94%. In PyPI, the average FPR of other tools in the OBF and RDE scenarios are 40.62% and 41.58%, respectively. In comparison, OSCAR's FPRs in these scenarios are 1.10% and 1.35%. This significant performance improvement is mainly attributed to our tool's dynamic execution and detailed API call capture and analysis mechanism.

Extracted from bhavyasoft-beta-4.5.8/package/src/structures/Product.js

```
1  .../*  This obfuscated code...
2  var _$_3c14=["\x75\x73\..."],...];_$_3c14[0];const Base=require
(_$_3c14[1]);const ProductMetadata=require(_$_3c14[2]);class Prod
uct extends Base{constructor(_0x91F7,_0x9234){...
```

Deobfuscation

```
1  var strs = ["use strict",  "./Base", ... ];
2  "use strict";
3  const Base = require("./Base");
4  const ProductMetadata = require("./ProductMetadata");
5  class Product extends Base {
6    constructor(data, productInfo) {
7      ...
8    }
9    _patch(productInfo) {
10     this.id = productInfo.id;
11     ...
12   }
13   ...
14 }
15 module.exports = Product;
```

Figure 4: JavaScript obfuscation sample.

Case#1: Obfuscation. As shown in Figure 4, the NPM package bhavyasoft-beta-4.5.8 contains multiple obfuscated `.js` files, including `Product.js`. After deobfuscating `Product.js`, we do not find any malicious behavior. However, as indicated in Table 7, the three rule-based tools match the relevant obfuscation rules, incorrectly marking the package as malicious. While SAP and Amalfi do not produce false positives for this package, they extract code entropy as a feature to target obfuscated malicious packages. However, these tools do not genuinely parse obfuscated code and cannot determine the actual behavior of the obfuscated code.

Table 7: Obfuscation sample instance detection results.

Artifact	Matched Rule	Key Log Info
AppInspector [23]	Hygiene: Suspicious Comment	`obfuscate`
OSSGadget [24]	Backdoor: Executing Obfuscated Code	`1\x69\x6C...`
Guarddog [2]	npm-obfuscation	`var _$_3c14=...`

Case#2: Remote Download and Execution. As shown in Figure 5, the `setup.py` file of the PyPI package ctcdecode contains both network requests (line 16) and command execution operations (line 27). However, these are merely preprocessing actions during the installation process and are not actually malicious. Yet, as shown in Table 8, the other tools merely detect the presence of these APIs without further analyzing how the code uses them. For example, a benign package might include code to download updates and start the update process. These tools, however, only see the network requests and process launches and mistakenly identify them as malicious behavior, leading to false positives.

Extracted from ctcdecode-1.0.2/setup.py

```
13 ...
14 def download_extract(url, dl_path):
15   if not os.path.isfile(dl_path):
16     urllib.request.urlretrieve(url, dl_path)
17   ...
18   tar = tarfile.open(dl_path)
19   tar.extractall('third_party/')
20   tar.close()
21
22 download_extract('https://github.com/...',
                   ↪'third_party/openfst-1.6.7.tar.gz')
23 # Does gcc compile with this header and library?
24 def compile_test(header, library):
25   adummy_path = os.path.join(os.path.dirname(__file__),
"dummy")
26   command = "bash -c \"g++ -include " + header + " -l" + ...
27   return os.system(command) == 0
28 ...
```

Figure 5: Python remote download and execution sample.

In contrast, OSCAR employs dynamic execution techniques, which are not affected by these risky but benign behaviors. For specific API calls, OSCAR capture their behavior and deeply analyze the parameters to determine their actual intent. For instance, OSCAR not only detect network requests and process launches but also further analyze the targets of these requests and the data being transmitted. This ensures that only genuinely malicious behavior is flagged. Consequently, OSCAR effectively reduces such false positives.

ANSWER to RQ2

OSCAR shows a significantly lower false positive rate compared to the other six static methods when detecting obfuscated and remote download and execution-like samples.

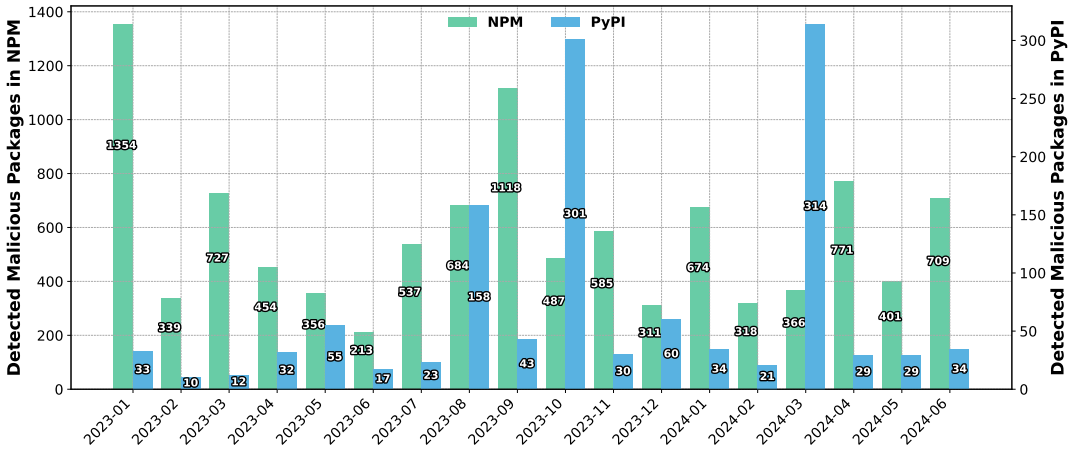


Figure 6: Malicious packages detected by OSCAR in NPM and PyPI over the past 18 months.

Table 8: RDE sample instance detection results.

Artifact	Matched Rule	Key Log Info
AppInspector [23]	Cloud Service: Code Repository	`download_extract ('https://git...`
Bandit4Mal [21]	tarfile_unsafe_members	`tar = tarfile. open(dl_path)...`
Guarddog [2]	code-execution	`return os.system (command)...`

5.5 Industrial Deployment (RQ3)

We have deployed OSCAR on Ant Group's NPM and PyPI mirrors and continuously monitored package updates for over 18 months, from January 2023 to June 2024. In total, we identify 10,404 malicious NPM packages and 1,235 malicious PyPI packages. Throughout this period, the detection system operates continuously, demonstrating its stability and robustness. It is important to note that due to the synchronization delay between mirror repositories and the official repository, packages that have a very short lifespan on the official repository and are not synchronized in time to the mirror repositories fall outside the scope of our detection. Consequently, there may be a significant discrepancy between the number of malicious packages actually uploaded to the official repository and those detected by OSCAR.

As shown in Table 9, detailed data analysis of classified malicious packages further reveals the distribution of different types of malicious behavior in the NPM and PyPI. Information gathering is predominant in the NPM, with 9228 such malicious packages detected, accounting for over 90% of the total detected. In the PyPI, information gathering also occupies a major portion, with 1,092 such malicious packages detected. Command execution has a high proportion in both repositories, indicating that attackers not only gather data but also attempt to execute remote commands through malicious packages to control infected systems. Although mining activities are relatively fewer in number, their high risk poses a serious threat to system performance and security. We also detected a small number of Proof-of-Concept (PoC) packages. Although fewer in number, these packages are particularly noteworthy as they are primarily created by security researchers for testing purposes. The presence of these PoCs indicates that the issue of open-source software supply chain attacks, particularly package poisoning, is receiving widespread attention from the research community. This growing focus underscores the increasing recognition of the importance of securing package ecosystems against potential threats.

Table 9: Unknown malicious classification.

Malicious Pattern	#Cnt (NPM)	#Cnt (PyPI)
Information Leakage	9228	1092
Command Execution	367	132
Proof-of-Concept	15	1
Cryptomining	14	1
Others	780	9
Total	10404	1235

Additionally, we conduct a monthly analysis of the detected malicious packages, as shown in Figure 6. We observe significant increases in the number of detected malicious packages during certain months. For example, in January and September 2023, the number of malicious packages detected in NPM reach 1,354 and 1,118, respectively. Similarly, in March 2024, the number of malicious packages in PyPI surge to 314. These peaks may be associated with specific events or large-scale attack campaigns, indicating heightened attacker activity during these periods. For example, in March 2024, attackers upload over a hundred malicious packages to PyPI, including packages like `requesqs` and `requetsts`, aiming to steal users' browser cookies, wallet credentials, and Discord tokens.

ANSWER to RQ3

OSCAR performs exceptionally well when deployed to real NPM and PyPI mirrors, successfully detecting a large number of malicious packages. This demonstrates its effectiveness and stability.

6 DISCUSSION

Implications. The significance of our research findings is highlighted in two key aspects. First, our work underscores the effectiveness of dynamic execution in reducing false positive rates in malware detection while maintaining high detection accuracy. We have demonstrated these points through extensive experiments. Second, our long-term monitoring of real storage mirrors of NPM and PyPI revealed a total of 11,639 malicious packages. This underscores the practicality of OSCAR and provides monthly data indicating the level of potential threats within these ecosystems. Additionally, during deployment, we continuously updated our pivot methods and heuristic matching rules to ensure the long-term effectiveness of our detection. Overall, these insights enhance malware detection and encourage future research, contributing to broader efforts to mitigate the impact of supply chain attacks.

Limitations. Despite the significant contributions of our research, there are some notable limitations. First, OSCAR primarily focuses on NPM and PyPI package managers, and has yet to explore other widely used package managers such as RubyGems, Maven, and .NET. Second, although OSCAR's malware detection accuracy is significantly improved compared to other methods, some scenarios remain uncovered: OSCAR's sandbox environment is Linux-based, so it currently cannot detect attacks targeting Windows systems; packages using anti-sandbox techniques to evade dynamic detection cannot be executed; and malicious code hidden in functions with complex parameters cannot be detected. However, we believe such malicious packages are rare due to stringent execution conditions, making it impractical to trigger the malicious logic. Lastly, despite OSCAR significantly reducing the false positive rate, it cannot completely eliminate false positives. Future work needs to address these limitations and further refine the detection of malicious code in the software supply chain.

7 RELATED WORK

Software Supply Chain Attacks. Given the critical role of the software supply chain in the computer ecosystem, various components of the software supply chain have consistently been targets of attacks, as discussed in recent works [1, 32, 34, 48]. In recent years, supply chain attacks targeting package managers have shown a significant upward trend, threatening the security of pre-built packages that facilitate code sharing [44, 46]. Previous studies have highlighted several challenges and threats in this domain. Enck et al. [4] summarized five key challenges facing OSS supply chain security. Ladisa et al. [15] provided a taxonomy of attacks across all stages of the OSS supply chain, from code contribution to package distribution, and assessed countermeasures against these attacks. Zimmermann et al. [50] conducted a systematic analysis of 609 known security issues, revealing the extensive attack surface within the NPM ecosystem. Guo et al. [8] performed an empirical study of supply chain attacks in the PyPI ecosystem, identifying a predominant single-function characteristic in PyPI poisoning incidents. Building upon these insights from prior research, we recognize that attackers typically design their malicious actions to be easily executable, which makes dynamic detection particularly effective for analyzing malicious packages. Leveraging this insight, our research advances the field by improving malicious package detection across ecosystems, while significantly reducing false positives in benign packages with risk-like features.

Malicious Code Poisoning Detection. Various methods have been proposed to detect malicious packages in software repositories, particularly focusing on NPM and PyPI ecosystems [16, 17, 29, 37, 45]. Vu et al. [47] proposed a rule-based method to detect malicious PyPI packages spread through typosquatting and combosquatting attacks. However, this method is ineffective against other types of attacks, revealing significant limitations. Duan et al. [3] introduced the MALOSS framework, which conducted a large-scale empirical study on the security of NPM, PyPI, and RubyGems. They derived five metadata analysis rules, four static analysis rules, and four dynamic analysis rules to detect malicious packages, discovering 339 previously undetected malicious packages. However, this method heavily relies on program analysis and is resource-intensive. In contrast to rule-based methods, Garrett et al. [7] selected features based on whether a package uses libraries to access the network, file system, and operating system processes, evaluates code at runtime, and creates new files. They used clustering to construct a benign behavior model to detect malicious NPM packages. However, this method captures malicious behavior as discrete features, which hinders its accuracy in detecting malicious packages. Zhang et al. [49] proposed CEREBRO, which organizes extracted features into behavior sequences, models continuous malicious behavior, and uses a fine-tuned BERT model to understand the semantics of malicious actions. In comparison with the above approaches, OSCAR fully executes packages in a sandbox environment, employs fuzz testing on exported functions and classes, and implements aspect-based behavior monitoring with tailored API hook points, allowing for more accurate detection of malicious behaviors while significantly reducing false positives in benign packages with risk-like characteristics.

8 CONCLUSION

This paper presented OSCAR, a dynamic code poisoning detection pipeline for NPM and PyPI ecosystems. By combining sandbox execution, fuzz testing, and aspect-based behavior monitoring, OSCAR achieves high accuracy in detecting malicious packages while significantly reducing false positives. Our evaluation demonstrates its effectiveness, with F1 scores of 0.95 for NPM and 0.91 for PyPI. In collaboration with Ant Group, a leading financial technology company, we deployed OSCAR on their NPM and PyPI mirrors. Over an 18-month period, this real-world industrial deployment successfully identified 10,404 malicious NPM packages and 1,235 malicious PyPI packages. This work not only bridges the gap between academic research and industrial application in code poisoning detection but also provides a robust, practical solution that has been thoroughly tested in a large-scale industrial setting.

ACKNOWLEDGEMENT

This work was supported by the National NSF of China (grants No.62302176, No.62072046, No.62302181), the Key R&D Program of Hubei Province (2023BAB017, 2023BAB079), the Knowledge Innovation Program of Wuhan-Basic Research (2022010801010083), Xiaomi Young Talents Program, and the research funding from MYbank (Ant Group).

REFERENCES

[1] Aarnav M. Bos. 2023. A Review of Attacks Against Language-Based Package Managers. arXiv:2302.08959 [cs.SE] https://arxiv.org/abs/2302.08959

[2] DataDog. 2024. GuardDog. https://github.com/DataDog/guarddog. Accessed: 2024-07-13.

[3] Ruian Duan, Omar Alrawi, Ranjita Pai Kasturi, Ryan Elder, Brendan Saltaformaggio, and Wenke Lee. 2021. Towards Measuring Supply Chain Attacks on Package Managers for Interpreted Languages. In *28th Annual Network and Distributed System Security Symposium, NDSS*. https://www.ndss-symposium.org/wp-content/uploads/ndss2021_1B-1_23055_paper.pdf

[4] William Enck and Laurie Williams. 2022. Top Five Challenges in Software Supply Chain Security: Observations From 30 Industry and Government Organizations. *IEEE Security & Privacy* 20, 2 (2022), 96–100. https://doi.org/10.1109/MSEC.2022.3142338

[5] falcosecurity. 2024. What is Malware & How to Stay Protected from Malware Attacks. https://github.com/falcosecurity/falco. Accessed: 2024-07-13.

[6] Python Software Foundation. 2024. The Python Package Index. https://pypi.org. Accessed: 2024-07-13.

[7] Kalil Garrett, Gabriel Ferreira, Limin Jia, Joshua Sunshine, and Christian Kästner. 2019. Detecting Suspicious Package Updates. In *2019 IEEE/ACM 41st International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER)*. 13–16. https://doi.org/10.1109/ICSE-NIER.2019.00012

[8] Wenbo Guo, Zhengzi Xu, Chengwei Liu, Cheng Huang, Yong Fang, and Yang Liu. 2023. An Empirical Study of Malicious Code In PyPI Ecosystem. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 166–177. https://doi.org/10.1109/ASE56229.2023.00135

[9] Cheng Huang, Nannan Wang, Ziyan Wang, Siqi Sun, Lingzi Li, Junren Chen, Qianchong Zhao, Jiaxuan Han, Zhen Yang, and Lei Shi. 2024. DONAPI: Malicious NPM Packages Detector using Behavior Sequence Knowledge Mapping. *arXiv preprint arXiv:2403.08334* (2024).

[10] Docker Hub. 2024. Node Docker Official Image. https://hub.docker.com/_/node. Accessed: 2024-07-13.

[11] Docker Hub. 2024. Python Docker Official Image. https://hub.docker.com/_/python. Accessed: 2024-07-13.

[12] Kevin Jesse, Premkumar T. Devanbu, and Toufique Ahmed. 2021. Learning type annotation: is big data enough?. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Athens, Greece) *(ESEC/FSE 2021)*. Association for Computing Machinery, New York, NY, USA, 1483–1486. https://doi.org/10.1145/3468264.3473135

[13] Maya Kaczorowski, Falcon Momot, George Neville-Neil, and Chris McCubbin. 2023. OSS Supply-Chain Security: What Will It Take? *Commun. ACM* 66, 4 (mar 2023), 48–54. https://doi.org/10.1145/3583119

[14] Gregor Kiczales, John Lamping, Anurag Mendhekar, Chris Maeda, Cristina Lopes, Jean-Marc Loingtier, and John Irwin. 1997. Aspect-oriented programming. In *ECOOP'97 — Object-Oriented Programming*, Mehmet Akşit and Satoshi Matsuoka (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 220–242.

[15] Piergiorgio Ladisa, Henrik Plate, Matias Martinez, and Olivier Barais. 2023. SoK: Taxonomy of Attacks on Open-Source Software Supply Chains. In *2023 IEEE Symposium on Security and Privacy (SP)*. 1509–1526. https://doi.org/10.1109/SP46215.2023.10179304

[16] Piergiorgio Ladisa, Serena Elisa Ponta, Nicola Ronzoni, Matias Martinez, and Olivier Barais. 2023. On the Feasibility of Cross-Language Detection of Malicious Packages in npm and PyPI. In *Proceedings of the 39th Annual Computer Security Applications Conference* (Austin, TX, USA) *(ACSAC '23)*. Association for Computing Machinery, New York, NY, USA, 71–82. https://doi.org/10.1145/3627106.3627138

[17] Ningke Li, Shenao Wang, Mingxi Feng, Kailong Wang, Meizhen Wang, and Haoyu Wang. 2023. MalWuKong: Towards Fast, Accurate, and Multilingual Detection of Malicious Code Poisoning in OSS Supply Chains. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 1993–2005. https://doi.org/10.1109/ASE56229.2023.00073

[18] Wen Li, Haoran Yang, Xiapu Luo, Long Cheng, and Haipeng Cai. 2023. PyRTFuzz: Detecting Bugs in Python Runtimes via Two-Level Collaborative Fuzzing. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security* (Copenhagen, Denmark) *(CCS '23)*. Association for Computing Machinery, New York, NY, USA, 1645–1659. https://doi.org/10.1145/3576915.3623166

[19] Chengwei Liu, Sen Chen, Lingling Fan, Bihuan Chen, Yang Liu, and Xin Peng. 2022. Demystifying the Vulnerability Propagation and Its Evolution via Dependency Trees in the NPM Ecosystem. In *2022 IEEE/ACM 44th International Conference on Software Engineering (ICSE)*. 672–684. https://doi.org/10.1145/3510003.3510142

[20] lxyeternal. 2024. pypi_malregistry. https://github.com/lxyeternal/pypi_malregistry. Accessed: 2024-07-13.

[21] lyvd. 2022. bandit4mal. https://github.com/lyvd/bandit4mal. Accessed: 2024-07-13.

[22] Luke Mcbride. 2021. Are you still wondering about dependency confusion attacks? https://www.sonatype.com/blog/are-you-still-wondering-about-dependency-confusion-attacks. Accessed: 2024-07-15.

[23] microsoft. 2024. ApplicationInspector. https://github.com/microsoft/ApplicationInspector. Accessed: 2024-07-13.

[24] microsoft. 2024. OSSGadget. https://github.com/microsoft/OSSGadget. Accessed: 2024-07-13.

[25] Amir M. Mir, Evaldas Latoškinas, Sebastian Proksch, and Georgios Gousios. 2022. Type4Py: practical deep similarity learning-based type inference for python. In *Proceedings of the 44th International Conference on Software Engineering* (Pittsburgh, Pennsylvania) *(ICSE '22)*. Association for Computing Machinery, New York, NY, USA, 2241–2252. https://doi.org/10.1145/3510003.3510124

[26] Marvin Moog, Markus Demmel, Michael Backes, and Aurore Fass. 2021. Statically Detecting JavaScript Obfuscation and Minification Techniques in the Wild. In *2021 51st Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. 569–580. https://doi.org/10.1109/DSN48987.2021.00065

[27] Aniruddhan Murali, Noble Mathews, Mahmoud Alfadel, Meiyappan Nagappan, and Meng Xu. 2024. FuzzSlice: Pruning False Positives in Static Analysis Warnings through Function-Level Fuzzing. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering* (Lisbon, Portugal) *(ICSE '24)*. Association for Computing Machinery, New York, NY, USA, Article 65, 13 pages. https://doi.org/10.1145/3597503.3623321

[28] npm. 2024. NPM Official Registry. https://registry.npmjs.org. Accessed: 2024-07-13.

[29] Marc Ohm, Felix Boes, Christian Bungartz, and Michael Meier. 2022. On the Feasibility of Supervised Machine Learning for the Detection of Malicious Software Packages. In *Proceedings of the 17th International Conference on Availability, Reliability and Security* (Vienna, Austria) *(ARES '22)*. Association for Computing Machinery, New York, NY, USA, Article 127, 10 pages. https://doi.org/10.1145/3538969.3544415

[30] Marc Ohm, Henrik Plate, Arnold Sykosch, and Michael Meier. 2020. Backstabber's Knife Collection: A Review of Open Source Software Supply Chain Attacks. In *Detection of Intrusions and Malware, and Vulnerability Assessment*, Clémentine Maurice, Leyla Bilge, Gianluca Stringhini, and Nuno Neves (Eds.). Springer International Publishing, Cham, 23–43.

[31] Marc Ohm and Charlene Stuke. 2023. SoK: Practical Detection of Software Supply Chain Attacks. In *Proceedings of the 18th International Conference on Availability, Reliability and Security* (Benevento, Italy) *(ARES '23)*. Association for Computing Machinery, New York, NY, USA, Article 33, 11 pages. https://doi.org/10.1145/3600160.3600162

[32] Marc Ohm, Arnold Sykosch, and Michael Meier. 2020. Towards detection of software supply chain attacks by forensic artifacts. In *Proceedings of the 15th International Conference on Availability, Reliability and Security* (Virtual Event, Ireland) *(ARES '20)*. Association for Computing Machinery, New York, NY, USA, Article 65, 6 pages. https://doi.org/10.1145/3407023.3409183

[33] Open Source Security Foundation (OpenSSF). 2024. package-analysis. https://github.com/ossf/package-analysis. Accessed: 2024-07-15.

[34] Ori Or-Meir, Nir Nissim, Yuval Elovici, and Lior Rokach. 2019. Dynamic Malware Analysis in the Modern Era—A State of the Art Survey. *ACM Comput. Surv.* 52, 5, Article 88 (sep 2019), 48 pages. https://doi.org/10.1145/3329786

[35] OSV. 2024. OSV Vulnerabilities Database. https://osv.dev/list?q=&ecosystem=npm. Accessed: 2024-09-13.

[36] Henrik Plate. 2020. OWASP Top 10 Risks for Open Source. https://www.endorlabs.com/learn/top-10-open-source-risks. Accessed: 2024-07-13.

[37] Adriana Sejfia and Max Schäfer. 2022. Practical automated detection of malicious npm packages. In *Proceedings of the 44th International Conference on Software Engineering* (Pittsburgh, Pennsylvania) *(ICSE '22)*. Association for Computing Machinery, New York, NY, USA, 1681–1692. https://doi.org/10.1145/3510003.3510104

[38] Philippe Skolka, Cristian-Alexandru Staicu, and Michael Pradel. 2019. Anything to Hide? Studying Minified and Obfuscated Code in the Web. In *The World Wide Web Conference* (San Francisco, CA, USA) *(WWW '19)*. Association for Computing Machinery, New York, NY, USA, 1735–1746. https://doi.org/10.1145/3308558.3313752

[39] Snyk. 2024. Snyk Vulnerability Database. https://security.snyk.io/vuln/npm. Accessed: 2024-09-13.

[40] Socket. 2024. alerts. https://socket.dev/alerts. Accessed: 2024-07-13.

[41] Socket. 2024. Known malware. https://socket.dev/alerts/malware/packages?ecosystem=npm&page=1. Accessed: 2024-09-13.

[42] Sonatype. 2024. Introducing our 9th annual State of the Software Supply Chain report. https://www.sonatype.com/blog/introducing-our-9th-annual-state-of-the-software-supply-chain-report. Accessed: 2024-07-13.

[43] Phylum Research Team. 2020. Typosquatting and Other Attacks Against Open Source Dependencies. https://blog.phylum.io/malicious-packages-typosquatting-and-other-attacks-against-open-source-dependencies/. Accessed: 2024-07-13.

[44] Ruturaj K. Vaidya, Lorenzo De Carli, Drew Davidson, and Vaibhav Rastogi. 2021. Security Issues in Language-based Software Ecosystems. arXiv:1903.02613 [cs.CR] https://arxiv.org/abs/1903.02613

[45] Duc-Ly Vu, Zachary Newman, and John Speed Meyers. 2023. Bad Snakes: Understanding and Improving Python Package Index Malware Scanning. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. 499–511.

https://doi.org/10.1109/ICSE48619.2023.00052

[46] Duc Ly Vu, Ivan Pashchenko, Fabio Massacci, Henrik Plate, and Antonino Sabetta. 2020. Towards Using Source Code Repositories to Identify Software Supply Chain Attacks. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security* (Virtual Event, USA) *(CCS '20)*. Association for Computing Machinery, New York, NY, USA, 2093–2095. https://doi.org/10.1145/3372297.3420015

[47] Duc-Ly Vu, Ivan Pashchenko, Fabio Massacci, Henrik Plate, and Antonino Sabetta. 2020. Typosquatting and Combosquatting Attacks on the Python Ecosystem. In *2020 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*. 509–514. https://doi.org/10.1109/EuroSPW51379.2020.00074

[48] Nusrat Zahan, Thomas Zimmermann, Patrice Godefroid, Brendan Murphy, Chandra Maddila, and Laurie Williams. 2022. What are weak links in the npm supply chain?. In *Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice* (Pittsburgh, Pennsylvania) *(ICSE-SEIP '22)*. Association for Computing Machinery, New York, NY, USA, 331–340. https://doi.org/10.1145/3510457.3513044

[49] Junan Zhang, Kaifeng Huang, Bihuan Chen, Chong Wang, Zhenhao Tian, and Xin Peng. 2023. Malicious Package Detection in NPM and PyPI using a Single Model of Malicious Behavior Sequence. arXiv:2309.02637 [cs.CR] https://arxiv.org/abs/2309.02637

[50] Markus Zimmermann, Cristian-Alexandru Staicu, Cam Tenny, and Michael Pradel. 2019. Small World with High Risks: A Study of Security Threats in the npm Ecosystem. In *28th USENIX Security Symposium (USENIX Security 19)*. USENIX Association, Santa Clara, CA, 995–1010. https://www.usenix.org/conference/usenixsecurity19/presentation/zimmerman

[51] Markus Zimmermann, Cristian-Alexandru Staicu, Cam Tenny, and Michael Pradel. 2019. Smallworld with high risks: a study of security threats in the npm ecosystem. In *Proceedings of the 28th USENIX Conference on Security Symposium* (Santa Clara, CA, USA) *(SEC'19)*. USENIX Association, USA, 995–1010.